学期助手（Semester Assistant）· 作品介绍

> 第九届传智杯 · AI 创新应用挑战赛（Vibe Coding 全栈应用速成）· 个人赛 · C 组 文档版本：预指导提交版 · 2026-10-25

一、项目详情与使用方式

1.1 它是什么

一个把课程资料、课表、作业截止时间放在同一条时间轴上的学习助手。

它做两件事：

1. **回答课程资料里的问题，并且每个答案都能点回原文页码**；如果资料里没有答案，它会明确说找不到，而不是编一个
2. **告诉你这周有什么课、哪份作业快到期**，把散落在课表和通知里的信息合成一张周视图

它不是一个通用文档问答工具。通用的知识库产品（如 AnythingLLM、RAGFlow）没有「今天是第几周」这个概念， 问不出「下周的数学作业要用哪一章」这类需要课表、作业、资料三者交叉的问题——这是本项目要填的空。

1.2 使用流程

> 启动 → 上传 PDF → 自动解析入库 → 提问 → 看答案与出处 → 切到「这周该做什么」

方式一：一键脚本（推荐）

```
.\setup.ps1
```

脚本依次安装后端依赖、构建前端、启动服务。浏览器打开 `http://127.0.0.1:8000`，左侧上传 PDF 并选择归属课程，主区提问。

方式二：手动四步

```
uv sync --default-index https://repo.huaweicloud.com/repository/pypi/simple
cd frontend; npm install; npm run build; cd ..
copy .env.example .env          # 填入 DEEPSEEK_API_KEY
uv run uvicorn backend.app.main:app --port 8000
```

方式三：命令行（用于验收与调试）

```
uv run python -m backend.cli index backend\samples\sample-lecture-notes.pdf   # 入库
uv run python -m backend.cli ask "标准差的含义是什么"                          # 问答
uv run python -m backend.cli week                                            # 本周课表与作业
```

1.3 一次典型交互

问：「习题 1.1 有哪些题目」

答：「习题 1.1 共 5 题：第 3 题 (4)、第 6 题、第 8 题 (3)、第 9 题 (2)(4) [1]。」 展开 `[1]`，看到《习题 1.1 画勾题解析.pdf》第 1 页的原文。

问：「习题 1.1 有没有难题」

答：「检索到了《习题 1.1 画勾题解析.pdf》第 1 页的相关内容，但里面没有直接回答这个问题的信息。最接近的原文在下面，你可以自己判断。」——资料里没有标注难度，它不会替学生下判断。

1.4 运行要求

- Python 3.12+、uv
- Node.js 18+（构建前端；产物由 FastAPI 直接托管，无需单独部署）
- 首次运行下载中文嵌入模型（约 100MB）与 PyTorch（Windows CPU 版），整个虚拟环境约 1GB
- 需要 DeepSeek API Key（脚本填入 `.env`，不进入代码与版本库）
- **全部计算在本机完成**，资料不外传，不做公网部署

二、项目亮点

2.1 时间轴 × 知识库（核心差异）

现成的知识库产品都是「静态语料 + 你问它答」。本项目的课表、作业、资料共用一条教学周时间轴：

- 第几教学周由开学日推算（第 1 周 = 2026-09-07）
- 本周课程来自真实课表（已脱敏，去除教师与教室）
- 作业是带截止日期的一次性事件，到期才出现在视图里
- 资料问答与周视图在同一个界面，问完资料可以直接看这周安排

2.2 可信度三态，而不是「有答案 / 没答案」

状态	表现
有依据	正常作答，每条结论带 `[n]`，可展开原文
相关但无答案	说明「检索到了《某资料》第 N 页，但里面没有直接回答这个问题的信息」，并把原文摆出来让用户判断
完全不相关	明说没检索到，并列出检索覆盖了哪些文档的哪些页

2.3 三层抗幻觉机制

1. **检索层**：最佳向量距离 > 0.65 直接返回「没检索到」，**不调用大模型**（省成本，也从源头杜绝编造）
2. **提示层**：system prompt 硬约束「只能用给定片段、每条结论标 `[n]`、没有依据只输出 `NO_ANSWER`」
3. **后处理层**：正则校验答案里的 `[n]` 是否真实存在；引用编号全部越界即判定为编造，整体降级为「找不到」

第三层是必须的：模型可能引用 `[7]`，而检索结果只有 5 条。

2.4 工程上的具体取舍（都有实测数据支撑）

- **切片不跨页**：双栏 PDF 跨页拼接会接上页脚和另一栏，且页码是唯一可回溯的坐标
- **切点优先落在句末**：纯按字符数硬切时实测 **92% 的片段断在句子中间**；改为「句末标点 > 空行 > 硬切」后，句末结尾率提升到 **80%**
- **嵌入模型必须按语言选**：默认英文模型 `all-MiniLM-L6-v2` 对中文完全失效（中文提问正确页距离 0.81，与「完全不相关」的 0.88 几乎无差别）；换成 `bge-small-zh-v1.5` 后正确页距离降到 **0.49**

三、AI 技术落地使用情况

3.1 用了哪些 AI 能力

环节	技术	说明
检索	BAAI/bge-small-zh-v1.5（本地 ONNX/PyTorch 推理）	中文语义检索，查询侧加官方指令前缀
生成	DeepSeek `deepseek-flash`（OpenAI 兼容协议）	温度 0 保证确定性；只依据检索片段作答；端点与模型名可在界面里改
开发	Claude Code	全程以「人类定义意图、AI 生成实现、人类验收纠错」的方式开发

3.2 AI 在开发中扮演什么角色

本项目的开发方式本身就是赛题要求的「Vibe Coding」：需求由人定义，代码由 AI 生成， 人负责验收、定位问题和给出约束。四个典型的纠偏过程都留了档（见 `docs/prompts/`）：

1. **解析与切片**：最初的切分 92% 断在句中，两轮修改断点策略才到 80%
2. **检索与模型选型**：默认英文模型对中文无效 → 换 BGE；发现混语言混库会互相污染
3. **问答与抗幻觉**：第一次测试返回「找不到」，排查后发现不是模型的问题，而是正确答案被挤出了前 5（距离差仅 0.03），把召回量从 5 提到 8 后解决
4. **学期视图**：把作业误建模成「每周固定条目」，被真实情况（开学前两周没作业）推翻，改为稀疏事件模型

每一步都是「AI 先给方案 → 跑出数据 → 判断哪里不对 → 给出新约束」。

原始对话已导出（脱敏）：`docs/prompts/raw/session-2026-10-25.md`，从项目正式启动（开始建仓库写代码）起记录，只保留对话文本，已去除密钥、UID、手机号、本地路径、学校与班级名称。

3.3 AI 使用的边界

- 大模型只负责「基于片段组织语言」，不负责决定答案对错；引用编号由后端校验
- 检索与嵌入全部本地运行，只有用户的问题和检索到的片段会发送给 DeepSeek
- API Key 只存在于本地 `.env`，不进入代码、日志和版本库

四、关键技术

模块	选型	关键实现
文档解析	PyMuPDF	逐页提取文本；`doc_id` 由文件内容哈希决定，重复上传幂等；无文本层（扫描件）明确报错而非静默入库
文本切片	自实现	按页切分不跨页；窗口后 40% 回退找句末/空行断点；600 字符、80 重叠（中文需降至 300–400）
向量库	Chroma（本地持久化）	集合名带模型标识，换嵌入模型即换集合，避免维度冲突
嵌入	bge-small-zh-v1.5	Chroma 1.x 的 `embed_documents` / `embed_query` 分开实现，查询侧加指令前缀
问答	DeepSeek + 引用校验	检索 → 编号上下文 → 生成 → 正则校验引用 → 三态返回
学期视图	纯规则计算	周次 = `(今天 - 开学日).days // 7 + 1`；作业为稀疏事件表；早于开学日返回空视图
界面	React 18 + TypeScript + Tailwind	`frontend/`；FastAPI 提供 REST 与 SSE 流式接口；出处卡片内嵌 PDF 原页渲染；组件与设计令牌部分取自 open-claude-ui（MIT），配色已整体替换

代码规模：后端 8 个模块约 1800 行（含 REST 接口层），前端约 1300 行（另有约 700 行取自 open-claude-ui 的组件），测试约 430 行，CLI 验收工具约 130 行。

五、项目图片

> 以下截图均为实际运行界面，数据使用仓库自带的示例讲义（无版权），不涉及任何个人信息。



图 1 资料问答：左侧为资料库、上传入口与「归属课程」，主区顶部为模型配置，下方为对话。答案后的 `[1]` 即出处标记；底部输入区可限定在某一门课内检索。

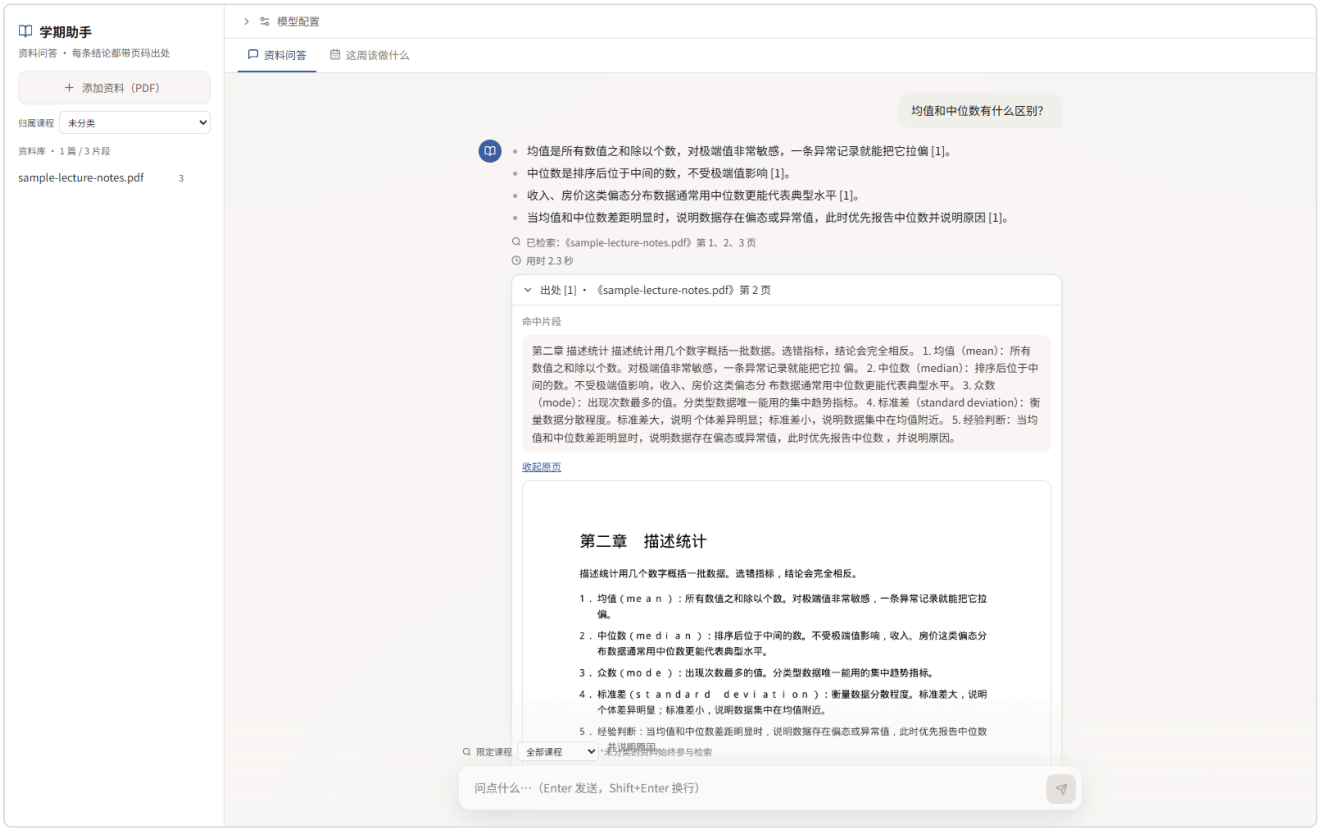


图 2 点开出处：上方是检索命中的原文片段，下方是 PDF 该页的真实渲染，两边可以逐字对照。多页文档会明确提示这是第几页、不是全文。

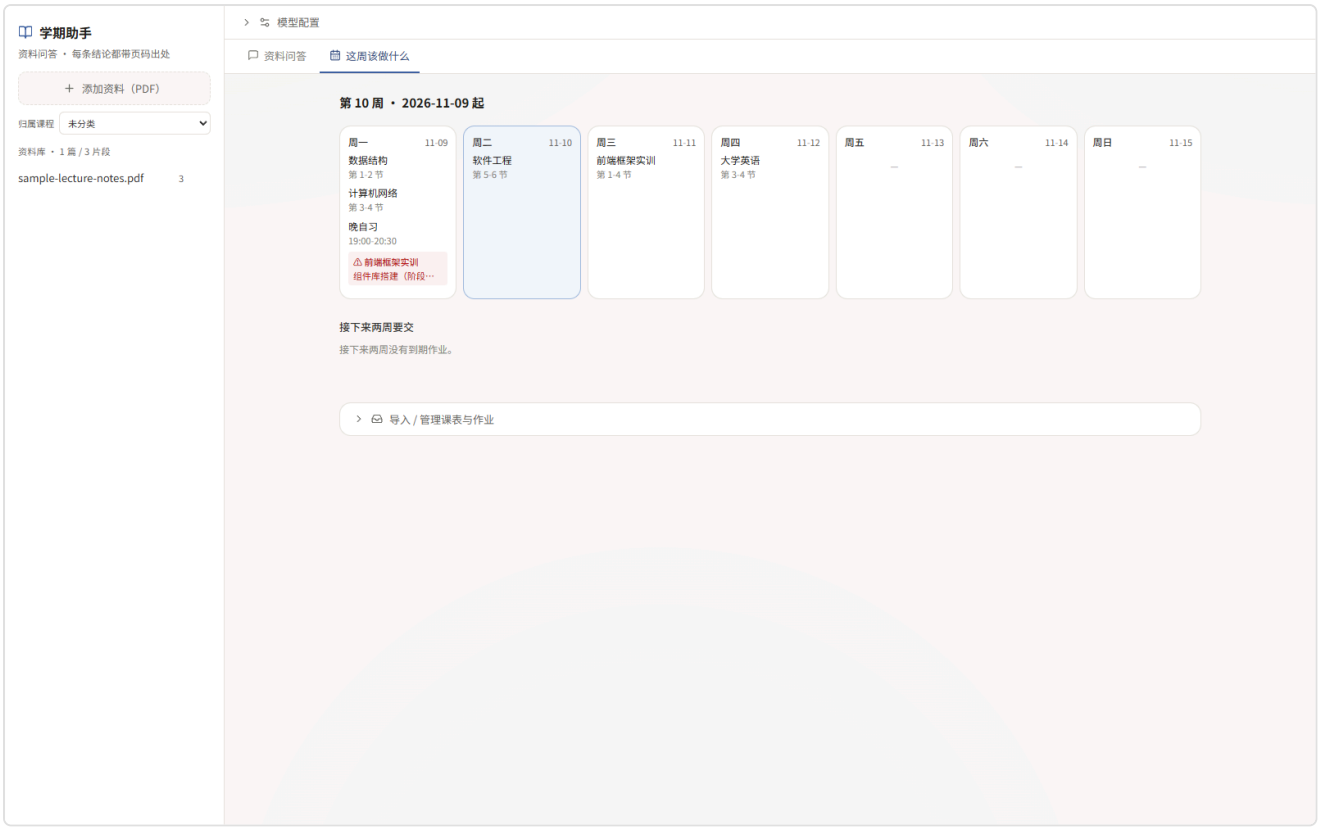


图 3 「这周该做什么」：当前教学周的课程安排，今天高亮，红色为当天到期作业。

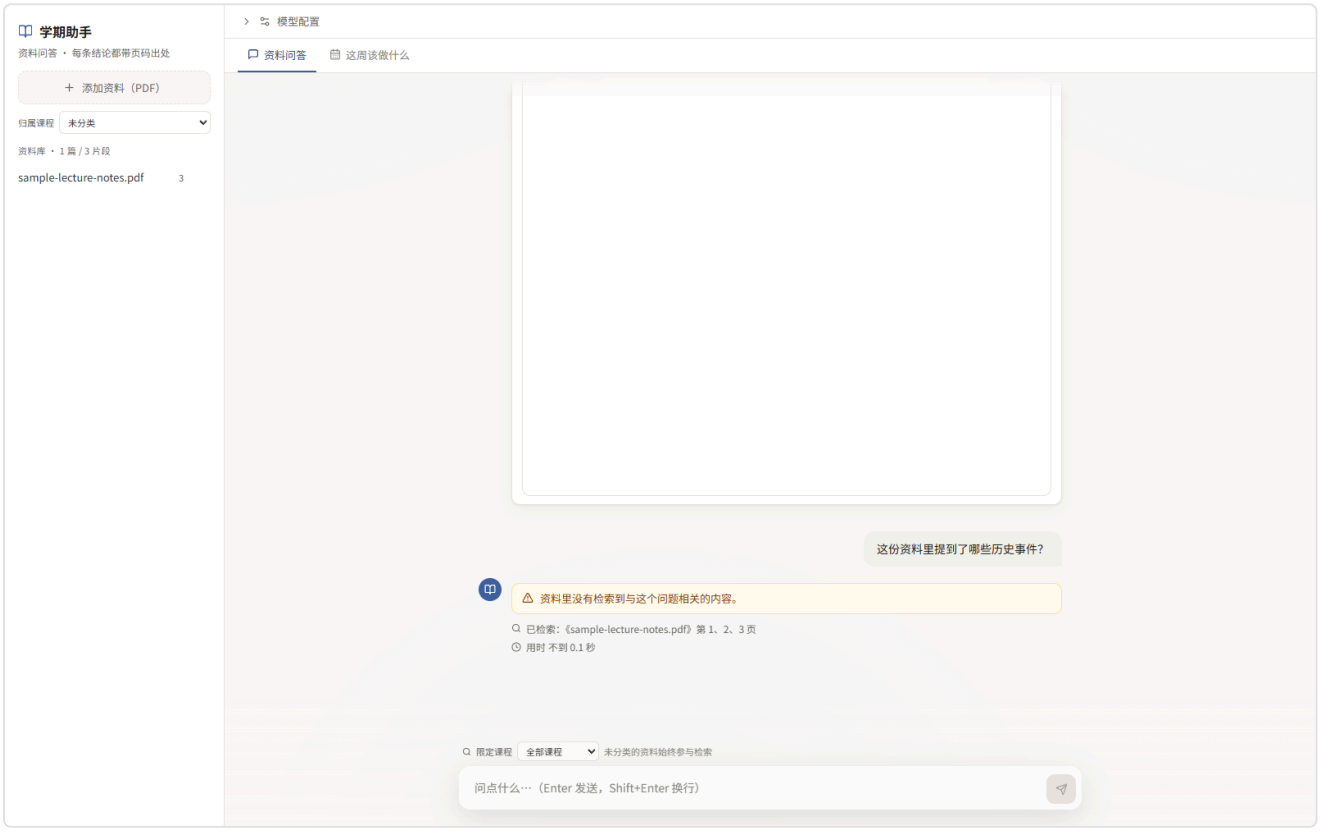


图 4 资料中没有相关内容时，明确说明检索到了什么、但没有直接答案，并把最接近的原文摆出来让用户自己判断，不编造答案。

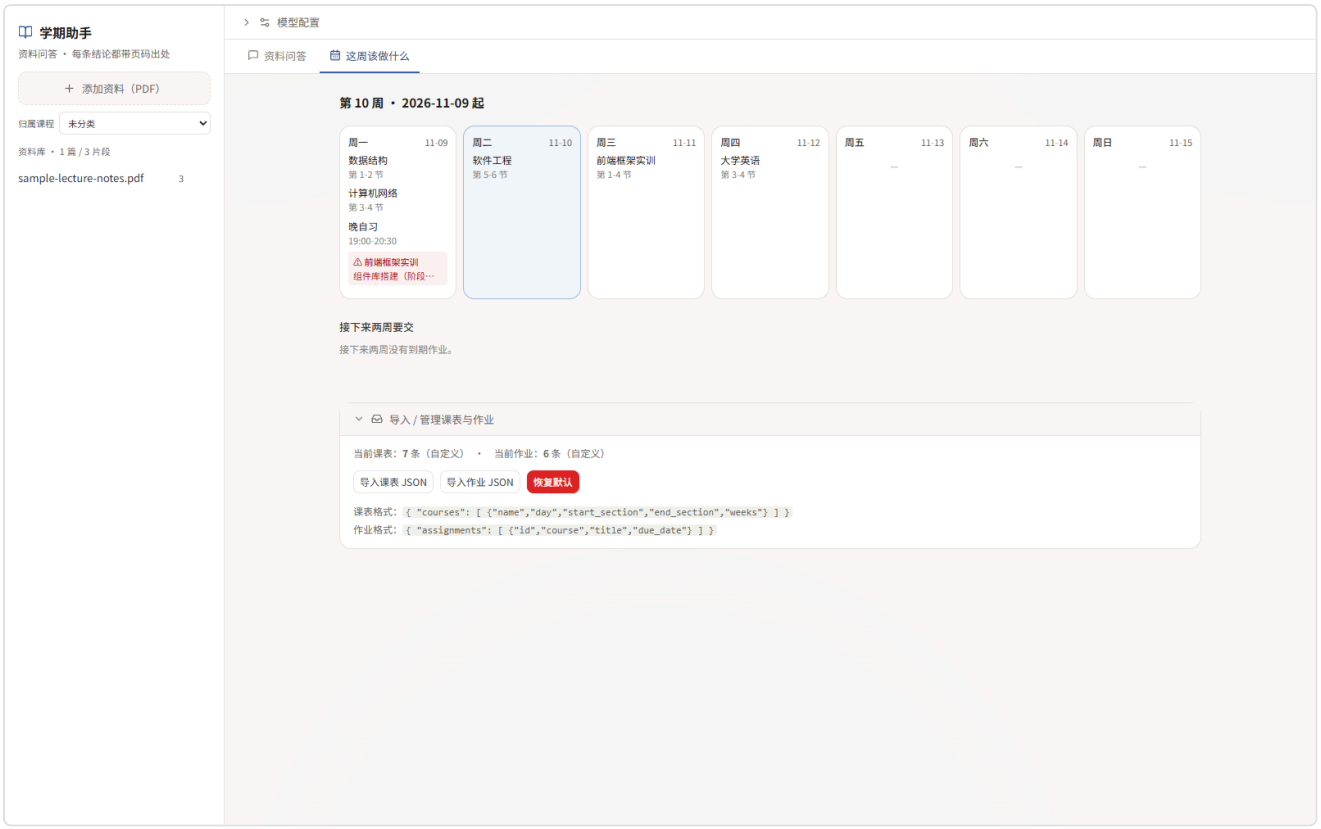


图 5 课表与作业可通过界面导入（JSON），支持恢复为仓库自带的示例数据；当前显示的是「接下来两周要交」与导入面板。



图 6 首次打开时的分步引导：依次高亮「添加资料」「提问输入区」「周视图」三个关键区域，每步一句话说明；右上角与气泡内都可跳过，目标元素定位失败时也有兜底出口，不会把用户锁在全屏遮罩里。

六、项目之前是否发表过或获奖

否。 本项目为本次比赛期间从零开发，此前未发表、未参加其他竞赛、未获得任何奖项。

七、是否在其他项目基础上优化

否。 本项目不是对任何开源项目的包装或二次封装。

说明用到的第三方依赖及其角色：

依赖	许可证	在本项目中的角色
PyMuPDF	AGPL-3.0	PDF 文本提取库
Chroma	Apache-2.0	本地向量数据库
BAAI/bge-small-zh-v1.5	MIT	预训练嵌入模型（非本项目训练）
React 18 / TypeScript / Tailwind CSS	MIT	前端框架与样式体系
open-claude-ui	MIT	部分 UI 组件与设计令牌（配色已改为本项目靛蓝，不复制其视觉）
DeepSeek API	商业服务	生成式大模型推理

项目自身的代码（解析调度、切片策略、引用校验、抗幻觉三层设计、学期视图、命令行工具）均为本次开发期间编写，Git 提交历史可查。

附：当前完成度与后续计划

已完成：解析、切片、入库检索、带引用问答（流式输出）、抗幻觉三层、学期周视图、按课程分组检索、React 前端（含课表导入、资料管理、PNG 原页对照）、命令行验收工具、后端 19 项 + 前端 14 项冒烟测试。

预指导后计划补强： 1. 补自动化冒烟测试 2. 检索质量优化（具体事实类问题的召回） 3. 按课程分组检索（当前混库时无关片段会互相干扰） 4. 更完整的界面（最终版考虑迁移到 React）